9. Interfaces and Composition

09. Interfaces and Composition

Phase: Phase 2 — Core language depth
Estimated time: 1–2 days
Slug: `09-interfaces`

Learning objectives

- Define interfaces as sets of methods
- Accept interfaces, return concrete types in APIs
- Compose behavior via small interfaces
- Understand the empty interface and `any`

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Interfaces are implicit

In Go you do **not** write `implements`. If a type has the methods, it satisfies the interface.

```
type Stringer interface {
    String() string
}

type User struct{ Name string }
func (u User) String() string { return u.Name }
```

Small interfaces win

`io.Reader`, `io.Writer`, `fmt.Stringer` — one or two methods. Write functions that take interfaces so callers can plug mocks, buffers, or network streams.

```
func CountLines(r io.Reader) (int, error) { /* ... */ }
```

Composition over inheritance

Go has no class hierarchy. Embed structs and combine interfaces instead.

```
type ReadWriter interface {
    io.Reader
    io.Writer
}
```

`any` (empty interface)

`any` is `interface{}` — every type satisfies it. Useful for containers before generics; prefer generics when you know the type set.

Common mistakes (especially from Python instincts)

- Defining huge interfaces that no one can mock
- Returning interfaces from constructors unnecessarily
- Using `any` everywhere instead of real types

Exercises

Work in `exercises/09-interfaces/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/09-interfaces/` until you have an honest failing attempt.

Self-check

1. How does a type satisfy an interface in Go?
2. Why are small interfaces preferred?
3. When is `any` acceptable?

Further reading

- https://go.dev/tour/methods
- https://go.dev/doc/effective_go#interfaces